

Quant Test

1. Introduction

The exercise has two halves, one for the hands and one for the head:

- **Part A — Manage a live paper trade.** Connect to Interactive Brokers, place a BTC futures trade or any other asset class you are allowed to trade in the demo account, and manage it in real time: Make a system that tracks Maximum Adverse Excursion (MAE) and closes the trade if it goes wrong early (Make the logic to determine what it means to go wrong early), and that adds another contract if the trade is working (Make the logic to determine what it means to be working)
- **Part B — Bring us one idea with edge.** Propose one BTC strategy, backtest it quickly, and measure it the way you would if your own money were behind it.

---

2. Ground rules

- Python preferred. Any libraries you like (`ib_async` / `ib_insync`, `pandas`, whatever).
- AI coding assistants are allowed. We use them too. What we evaluate is your judgment and code understanding.
- You will need an IBKR account with **paper trading** enabled and permissions on the paper account.

---

3. Part A — Manage a live paper trade on IBKR

The task

Build a small program that connects to your IBKR paper account, enters a position in **CME Bitcoin futures (symbol `BRR`, front month)**, or any other asset, and then *manages* it using two rules:

1. **Cut losers fast, using MAE.** Track the trade's Maximum Adverse Excursion in real time, expressed in **R** (R = your initial risk per unit, i.e. entry-to-initial-stop distance). If the trade goes against you beyond a threshold you define *before* the trade goes on, close it early — don't wait for the full stop.
2. **Add to winners.** If the trade is working (define "working" — e.g. MFE reaches +1R, or price holds beyond some level), add size, with explicit limits on how many times and how much. Decide what happens to the stop after an add.

You may use `MBT` (micro BTC, 0.1 BTC per contract) instead of `BRR` (5 BTC per contract) if you need finer sizing granularity for the add logic. If you do, say why in one line.

The entry trigger is not the point of Part A. A dead-simple trigger (breakout of the last N minutes, or even a manual "go long now" command) is fine. **The trade management is the point.**

Required

- Connect to TWS / IB Gateway paper, resolve the front-month contract programmatically (not hardcoded expiry).
- Place the entry with an attached initial stop (bracket or equivalent). Explain briefly why you chose the order types you did.
- While in the trade, continuously compute and log: current P&L in R, MAE (in R and in price), MFE, time in trade, and current size.
- Implement your MAE early-close rule and your add rule. Both parameters must be written down *before* the trade is live, with a one-paragraph justification for each number.
- Write a **trade journal** to disk (CSV or JSONL): every decision (entry, add, early-close, stop move) with timestamp, price, size, MAE/MFE at that moment, and the rule that fired.
- Run at least one real paper trade end-to-end and include the journal in your submission. We'll ask you to run it again live in front of us.

Bonus (only if the required part is solid)

- Reconcile state on restart: if your script dies mid-trade and you relaunch it, it should recover the open position and stop from IBKR rather than assume a clean slate.

- Handle partial fills correctly in your R and MAE calculations.
- A kill switch (flatten everything, cancel all orders) reachable in one command.
- Derive the MAE threshold from data rather than intuition.
- Propose Risk Management techniques that you have an edge in and provide the argument using data.

---

4. Part B — Bring us one idea with edge

The task

Propose **one** BTC trading strategy you believe has real edge. Any timeframe, any style — trend, mean reversion, basis, funding, volatility, microstructure, on-chain, seasonality. Free choice. Then test it quickly and measure it honestly.

We are far more interested in *why the edge exists* and *how you measured it* than in the equity curve itself.

Required

1. The thesis (half a page, plain language).
Why does this inefficiency exist? Who is on the other side of the trade, and why are they willing to lose to you? What would make the edge disappear?

2. A quick backtest.
Use any free BTC data you can get in the time (Binance / Coinbase public klines, Kaggle, CSVs — anything; note basis differences vs. CME if you think they matter). Keep it simple: vectorised pandas is fine, no need for an event-driven engine.

3. Honest measurement. At minimum:

- Number of trades, hit rate, average win / average loss (payoff ratio), expectancy in R.
- Sharpe (state the annualisation), max drawdown, drawdown duration.
- A cost assumption (fees + slippage) and how the results change if you double it.
- An in-sample / out-of-sample split, and a walk-forward correlation test. State what you looked at before you fixed the parameters.

- MAE and MFE distributions of winners vs. losers.
- Sensitivity of results to your main parameter(s). A strategy that only works at one setting is a curve fit.

4. The graveyard.
List the variants or ideas you tried in these 24 hours that *didn't* work, in a few lines each. This is not optional, and it is not a weakness.

5. Live monitoring.
If this were running with real money: what two or three numbers would you watch daily, and at what values would you declare the edge dead and switch it off?

Part C — Design the systematic trading intelligence stack

> Available inputs include:
>
> - OHLCV
> - trades/order book
> - CME futures
> - funding rates
> - perpetual futures
> - options/implied volatility
> - macro data
> - news
> - social/text data
> - existing strategy performance
>
> For each part of the system, decide whether you would primarily use:
>
> - deterministic/hard-coded rules,
> - classical statistical models,
> - tree-based ML such as Random Forest/GBM,
> - deep learning,
> - LLM agents,
> - or no model at all.

5. Connect A and B

Replicate the backtesting logic in a real-time system. Measure that both the backtest and the real-time system took the same trades during the evaluation period.

---

6. Deliverables

Send us, a link to a repository (GitHub/GitLab, private is fine — we'll give you handles to add) containing:

1. `README.md` — how to run Part A in under 5 minutes; assumptions; what you cut and why.
2. Part A code and at least one **trade journal** from a real paper run.
3. Part B notebook or script plus a short write-up (markdown or the notebook itself) covering items 1–5.
4. Trading Journal of the backtest and the Real-Time trades for the same period.
5. `NOTES.md` — anything you'd tell a colleague picking this up tomorrow: known bugs, what you'd do with one more day.

---

7. The presentation (45 minutes)

- **15 min — Live demo of Part A.** Run your bot against paper in front of us, put on a trade, and let us watch it manage.
- **10 min — Part B.** Thesis, results, graveyard. And the replication monitor for the backtest vs. real-time trades.
- **20 min — Questions.** We'll go deep on decisions, not on syntax.

---

8. Practical notes

IBKR setup. Paper accounts are created from your IBKR client portal; enable futures trading permissions on the paper account (this can take up to a day — do it as soon as you read this). Run TWS or IB Gateway in paper mode; default

API ports are 7497 (TWS paper) and 4002 (Gateway paper). Enable API connections in settings.

Market data. Live CME crypto data needs a subscription; **delayed data is acceptable** for this exercise (`reqMarketDataType(3)` in ib_async/ib_insync). Say in your notes which you used and how it affects the MAE tracking. Or use other asset if you have problems.

Contract. CME Bitcoin futures: `BRR` (5 BTC/contract), micro: `MBT` (0.1 BTC/contract). Resolve the front month via contract details rather than hardcoding. Note CME crypto futures do not trade 24/7 — check the hours and the daily maintenance break; we will send this exercise on a day when the market is open.

Paper account balance. Default paper equity is typically $1M. Size as if it were a $100k account — say what risk per trade you chose.

---

9. What we're looking for

- **Broker fluency.** You've clearly wired a broker API before: contract resolution, order state, fills, reconnects — things that only bite once you've been bitten.
- **Trade management judgment.** Your MAE and add rules have reasons behind them, and you understand their side effects (what an add does to your average price, your stop distance, your risk).
- **Measurement discipline.** You know the difference between a backtest and a result. You show what you tried that failed.
- **Speed with honesty.** You cut scope explicitly, prioritise well, and can explain every line you shipped.
- **Clear communication.** Someone on our team could pick up your repo tomorrow.

We are not looking for a perfect equity curve, a large codebase, or a finished product.

---

10. FAQ

Can I use a different broker / venue? Yes — IBKR is what we use, but another is acceptable.

Can I use a strategy I've already built? Yes, if it's yours. Tell us it's pre-existing, and spend the saved time on measurement quality.

What if the market is closed or my data feed breaks during the demo? Have a replay mode or a way to simulate ticks so the management logic can still be shown. Bonus points, not a requirement.